

Static code analysis tools

Apart from the performance analysis integrated in the proprietary embedded tools, there are certain third-party tools which can be seamlessly integrated into your IDEs or your team's natural workflow and analyse your codes.

Static code analysis tools like Coverity and Klockwork have emerged with a great impact on the sales and product integrity. Experts believe that if there is a tool which could report the issue before it actually causes an outage, you should have that tool.

Dhaval Vasa, product manager, elfin chips explains, "You can use these third-party tools as a part of your development environment, and then get things done. The static code analysis enables you to analyse your code before you actually combine it and make sure you remove all possible issues that exist in terms of security and second on the possible leakages that exist on the resource and the memory side. It is always a great help."

These tools can mirror how code is developed and find vulnerabilities on the run and thereby prevent any defects, as code is being written. You are thus enabled to take corrective actions immediately, before builds, and deliver more secure and reliable codes.

Tools are getting equipped with more and more coding standards, providing comprehensive defect coverage for various industries like automotive, embedded security, medical, aerospace and even gaming and mobile device developments.

software debug and Veloce Emulator allow hardware-software co-debug. You can use Veloche GUI to download hardware design and Sourcery Codebench Virtual Environment for software development and then debug through a virtual JTAG probe connected to the processor.

Embedded design suite from Xilinx also comes with the software/hardware cross feature, enabling software and hardware teams to debug in the same environment and allowing each team to capture both software and hardware data without having to use or learn separate tools.

Reduced bring-up time

For software validation in context of system hardware, methodologies like prototyping are widely used, and FPGA-based prototyping has surfaced with the solutions. But there are certain associated challenges.

The bring-up time to implement a design into an FPGA prototype has become a major challenge these days. It is not easy to make changes to the register-transfer level (RTL) design. Also, there hasn't been an easy transition available from an existing simulation or emulation environment to the prototype.

Same bring-up flow for emulation and rapid prototyping enables designers to switch seamlessly be-

tween two execution engines, which in-turn reduces the prototype bring-up time significantly as compared to traditional FPGA-based prototyping approaches.

The Protium platform from Cadence, based on Xilinx Virtex-7 2000T FPGAs and featuring an advanced implementation and debug software flow, addresses the above challenges. Common compile flow between Palladium emulation and Protium FPGA-based prototyping allow designers to have a quick and efficient transition from emulation to prototyping.

The platform supports up to 100 million gates, which is a 4X increase

in capacity compared to the first-generation Rapid Prototyping Platform. A fully automatic software flow enables fast prototype bring-up while additional user-driven performance optimisations assure highest possible speed, which is essential for early software development.

Power aware feature

This feature can be seen as a part of new performance analysis features that were mentioned earlier. In-depth analysis of performance and events of CPU and GPU activities allow identification of wake-up causes, timers triggered for different applications and various interrupts. Now you can maximise power efficiency of the silicon using functions provided in software tools.

Precision 32 from Silicon Labs enables developers to maximise power efficiency of embedded 32-bit designs. It features a power estimation view where the relative power profile of any given configuration can be seen and analysed. You can then make optimisations to the configuration using the tool and improve overall power consumption of the system.

Increasing hardware and software complexity in SoC designs has made low-power designing even more challenging. Juergen Jaeger, senior manager, product management and

IP builder for FPGAs

The concern for any FPGA designer is not only to run the code on the hardware but also to ensure the timing constraint, that is, whether it will run in the clock cycle supplied to the chip. Instead of using low-level hardware description languages, tools are giving us options to generate programs for FPGAs using simple languages like ANSI C or C++.

This helps separate the performance and space constraints from the code and allows any designer to focus only on performance of the code rather than other constraints.

Arun Veeramani from embedded systems segment, National Instruments, says, "For any designer, there are three main constraints which should be taken care of: performance, timing and resource constraints. The ideal situation would be to focus on performance and leave timing and resource constraints for the software to manage. There has been a lot of focus on high-level synthesis in last one year."

LabVIEW for FPGAs has introduced an IP builder feature for such high-level synthesis. It allows you to build almost anything and create an IP which will automatically optimise your FPGAs and achieve performance and resource utilisation goals.

Also, you can minimise code changes between your LabVIEW desktop and FPGA algorithm implementation, allowing a quick check for design trade-offs, without any need to change your algorithm.